

CS-553 Final Exam

Take Home - Summer 2026

Schrimpscher

This is an applied take-home exam, not a pile of trivia questions.

Provided Express Project: <https://github.com/UAH-Computer-Science/cs453-final-project>

- Create your own private repository from the provided starter project. Do not submit changes or pull requests to the shared starter repository. If you download the starter as a ZIP, extract it into your own repository before beginning.
- If you submit a **private** GitHub repository, ensure that the instructor can access it. Otherwise, submit a ZIP file.

Primary Coverage: databases, authentication, authorization, PKI, Certificates, JWTs, OAuth, messaging, asynchronous processing.

Secondary Coverage: Sockets, HTTP/JSON, REST-style resource design, Express routes, middleware, basic clients, and OpenAPI.

Submission: Submit a GitHub repository link or ZIP file containing:

1. `answers.md`
2. JavaScript source files for Parts 3–5
3. `openapi.yaml`
4. The provided `README.md`, updated with any additional setup or run instructions required by your implementation

Implement Parts 3–5 in the provided Express project. Clearly identify the relevant files in your README.

Allowed resources: Course notes, slides, labs, official Node/Express/OpenAPI documentation.

Not allowed: Collaboration with classmates, copying from online solutions, or using AI tools to generate code or written answers. AI may be used only as a study aid for understanding concepts, consistent with the course AI policy.

Part 1 — Conceptual Foundations

20 points

Answer each question in `answers.md`. Use complete sentences, but do not write a novel. A strong answer should explain the idea clearly and use appropriate technical terminology.

1. Authentication vs. Authorization

Explain the difference between authentication and authorization. Then describe what an API should return in each situation:

- The request does not contain valid authentication credentials.
- The caller is authenticated but does not have permission to perform the requested operation.

Include the appropriate HTTP status code for each situation.

3 points

2. Passwords, Sessions, and Tokens

An application allows users to log in with a username and password.

Explain:

- why the application should never store passwords as plain text,
- what the server should store instead,
- how a session-based login differs from a token-based login,
- one advantage of each approach.

3 points

3. JSON Web Tokens

Describe the purpose and structure of a JSON Web Token (JWT). Your answer should include:

- the three major parts of a JWT,
- the difference between signing and encrypting a token,
- why a server must validate a JWT before trusting its claims,
- one risk of using JWTs with excessively long expiration times.

3 points

4. OAuth

Explain the purpose of OAuth. Your answer should distinguish among:

- the resource owner,
- the client application,
- the authorization server,
- the resource server,
- the access token.

Also explain why giving a third-party application an OAuth access token is safer than giving it the user's password.

3 points

5. PKI and Certificates

Explain how a digital certificate helps a client establish a secure connection to an API server. Your answer should include:

- the purpose of the server's public and private keys,
- the role of a certificate authority,
- what the client verifies in the certificate,
- what could happen if certificate validation were skipped.

3 points

6. Databases, Messages, and Asynchronous Processing

An API receives a request to generate a large report. Producing the report may take several minutes.

Explain why the API should normally use asynchronous processing instead of keeping the HTTP request open. Describe a reasonable design that includes:

- a database record representing the requested job,
- a message queue,
- a background worker,
- an immediate HTTP response,
- a way for the client to check the job's status.

Include the successful HTTP status code for submitting the job and the successful status code for retrieving its current status.

5 points If you submit a private GitHub repository, ensure that the instructor can access it. Otherwise, submit a ZIP file.

Part 2 — Secure API Design

15 points

You are designing an API for a **Course Task Tracker**. Users authenticate through the university's OAuth service. After login, the API receives a signed JWT containing the user's identity and role.

The system supports two roles:

- **Student** — may view and update their own tasks.
- **Instructor** — may view and update any task.

Users may also request a progress report. Reports take several minutes to generate, so the system processes them asynchronously using a message queue and background worker.

A task stored in the database has the following representation:

```
{
  "id": "42",
  "title": "Watch Week 6 lecture",
  "course": "CS453",
  "studentId": "djs001",
  "completed": false
}
```

A report job has the following representation:

```
{
  "id": "report-17",
  "studentId": "djs001",
  "status": "pending",
  "downloadUrl": null
}
```

In `answers.md`, provide the following design.

Note: Do not repeat the general definitions from Part 1. Apply the concepts specifically to this API.

1. Authentication and Authorization

For each request below, state whether it should be allowed or rejected. If it is rejected, provide the appropriate HTTP status code.

Request	Decision and Status Code
A request contains no access token	
A request contains an expired JWT	
A student requests one of their own tasks	
A student requests another student's task	
An instructor requests a task belonging to any student	

Briefly explain where authentication ends and authorization begins when processing these requests.

5 points

2. OAuth, JWT, and PKI Design

Describe how the API should use OAuth, JWTs, and PKI when handling a request. Your design should identify:

- who issues the access token,
- how the client sends the token to the API,
- what the API must validate before trusting the JWT,
- how HTTPS and the server's certificate protect the connection,

- why the API must not trust a role supplied in the request body.

5 points

3. Database and Asynchronous Report Processing

Design the report-generation portion of the API. Provide:

- a method and URI for requesting a new report,
- the database record created for the report job,
- the message placed on the queue,
- the immediate HTTP status code and response body,
- a method and URI for checking the report's status,
- the changes made by the background worker when processing succeeds or fails.

Your design must not keep the original HTTP request open while the report is generated.

5 points

Part 3 — Authentication and Authorization Implementation

20 points

The Course Task Tracker uses JWT access tokens. Each valid token contains the following claims:

```
{
  "sub": "djs001",
  "role": "student"
}
```

The `sub` claim identifies the user. The `role` claim is either `"student"` or `"instructor"`.

Complete the authentication and authorization middleware in `src/middleware/auth.js`. The project has already imported `jsonwebtoken` as `jwt` and provides the configured secret as `config.jwtSecret`.

1. Authentication Middleware

Implement an Express middleware function named:

```
function authenticateToken(req, res, next) {
  // Your implementation
}
```

The middleware must:

- read the `Authorization` request header,
- require the format `Bearer <token>`,

- return `401 Unauthorized` if the token is missing or malformed,
- verify the token using `jwt.verify()` and `config.jwtSecret`,
- return `401 Unauthorized` if the token is expired or invalid,
- store the verified token payload in `req.user`,
- call `next()` when authentication succeeds.

8 points

2. Role-Based Authorization Middleware

Implement a middleware factory named:

```
function requireRole(...allowedRoles) {  
  // Your implementation  
}
```

The middleware returned by this function must:

- assume that `authenticateToken` has already run,
- examine `req.user.role`,
- call `next()` if the user has one of the allowed roles,
- return `403 Forbidden` if the user is authenticated but does not have an allowed role.

6 points

3. Protecting Routes

Apply the middleware to the following routes:

```
tasksRouter.get(  
  "/",  
  // Replace part3NotImplemented with the required middleware.  
  part3NotImplemented,  
  (req, res) => {  
    res.json({  
      userId: req.user.sub,  
      tasks: []  
    });  
  }  
);  
  
tasksRouter.delete(  
  "/:id",  
  // Replace part3NotImplemented with the required middleware.  
  part3NotImplemented,  
  async (req, res, next) => {  
    // The supplied deletion handler follows here.
```

```
}  
);
```

Do not rewrite the supplied route handlers. Replace `part3NotImplemented` with the appropriate authentication and role-authorization middleware.

The routes must follow these rules:

- Both students and instructors may access `GET /tasks`.
- Only instructors may access `DELETE /tasks/:id`.
- Authentication must occur before role authorization.

4 points

4. Error Classification

In `answers.md`, identify whether each situation should return `401 Unauthorized` or `403 Forbidden`.

Situation	Status Code
No access token was provided	
The JWT has expired	
The JWT signature is invalid	
A validly authenticated student attempts an instructor-only operation	

2 points

Part 4 — Database Integration and Async/Await

10 points

The Course Task Tracker stores tasks in a SQLite database. The provided project has already created and seeded the database.

The `tasks` table contains:

Column	Description
<code>id</code>	Unique task identifier
<code>title</code>	Task title
<code>course</code>	Course identifier
<code>student_id</code>	ID of the student who owns the task
<code>completed</code>	<code>0</code> for false or <code>1</code> for true

The provided database object supports asynchronous parameterized queries:

```
const result = await db.query(sql, parameters);
```

The selected rows are returned in:

```
result.rows
```

Assume that the authentication middleware from Part 3 stores the verified JWT payload in:

```
req.user
```

For example:

```
{
  "sub": "djs001",
  "role": "student"
}
```

1. Retrieve a Task

Complete the `GET /tasks/:id` route in `src/routes/tasks.js`.

Replace `part3NotImplemented` with the appropriate authentication and role-authorization middleware, and complete the route handler.

The route must:

- authenticate the request,
- allow users with the `"student"` or `"instructor"` role to reach the handler,
- retrieve the requested task using `req.params.id`,
- use a parameterized SQL query,
- return `404 Not Found` if the task does not exist,
- allow an instructor to retrieve any task,
- allow a student to retrieve a task only when its `studentId` matches `req.user.sub`,
- return `403 Forbidden` when a student requests another student's task,
- return `200 OK` with the task as JSON when successful,
- pass unexpected database errors to Express error-handling middleware using `next(error)`.

Use a SQL alias so that the database column `student_id` appears in the result as `studentId`.

SQLite represents `completed` as `0` or `1`. Convert it to a JavaScript Boolean before returning the task:

```
completed: Boolean(task.completed)
```

8 points

2. Database and Asynchronous Behavior

In `answers.md`, briefly answer both questions:

1. Why should the task ID be supplied as a query parameter instead of being inserted directly into the SQL string?
2. Why must the route use `await` when calling `db.query()`?

2 points

Part 5 — Message Queues and Background Processing

10 points

Generating a progress report may take several minutes. The API must create a job, place a message on a queue, and return a response without waiting for the report to finish.

The queue, database, and report generator have been provided for you.

Important

The queue infrastructure is already configured. You are responsible only for completing the route and worker logic shown below. Valid JavaScript is required; no additional queue configuration is necessary.

Complete the report-submission route in `src/routes/reports.js` and the queue worker in `src/workers/reportWorker.js`. All required imports, queue configuration, database helpers, and worker registration have already been provided.

The provided objects support these operations:

```
// Inserts a report job into the database.
await db.createReportJob({
  id,
  studentId,
  status,
  downloadUrl
});

// Changes fields in an existing report job.
await db.updateReportJob(jobId, {
  status,
  downloadUrl
});

// Adds a message to the already-configured queue.
await reportQueue.send({
  jobId,
  studentId
});

// Registers a function that processes messages from the queue.
// The queue calls the function whenever a message is available.
reportQueue.process(async (message) => {
```

```
// Process message
});

// Generates the report and returns its download location.
const downloadUrl = await generateReport(studentId);
```

Important

Do not modify `src/reportQueue.js`, `src/reportGenerator.js`, or the supplied database helper functions.

1. Submit a Report Job

Complete the marked portion of the route:

```
reportsRouter.post(
  "/",
  authenticateToken,
  async (req, res, next) => {
    try {
      const jobId = randomUUID();
      const studentId = req.user.sub;

      // Complete the TODOs in the provided project.
    } catch (error) {
      return next(error);
    }
  }
);
```

The completed route must:

- create a database job with a status of `"pending"`,
- place the job ID and student ID on the queue,
- return immediately without calling `generateReport`,
- return `202 Accepted`,
- return the job ID, status, and status URI as JSON.

An appropriate response has this form:

```
{
  "jobId": "report-17",
  "status": "pending",
  "statusUrl": "/reports/report-17"
}
```

5 points

2. Process a Report Message

Complete the queue worker:

```
reportQueue.process(async (message) => {
  const { jobId, studentId } = message;

  // TODO:
  // 1. Mark the job as processing.
  // 2. Generate the report.
  // 3. Mark the job as completed and save its download URL.
  // 4. If generation fails, mark the job as failed.
});
```

The worker must:

- change the job status to "processing" before generating the report,
- call `generateReport(studentId)`,
- change the status to "completed" after success,
- save the returned download URL,
- catch report-generation errors,
- change the job status to "failed" when an error occurs.

4 points

3. Queue Behavior

In `answers.md`, answer the following in two or three sentences:

- why the API returns `202 Accepted` instead of `200 OK` or `201 Created`,
- one advantage of generating the report in a background worker instead of inside the route handler.

1 point

Part 6 — OpenAPI Specification

15 points

Create an `openapi.yaml` file documenting the Course Task Tracker API.

```
GET    /tasks
GET    /tasks/{id}
POST   /tasks
PUT    /tasks/{id}
PATCH /tasks/{id}
DELETE /tasks/{id}
POST   /reports
GET    /reports/{id}
```

Important

You only have to **document** all eight routes; you only have to implement the routes required in Parts 3-5.

Also use `{id}` when discussing OpenAPI path parameters, not `:id`. Express uses `:id`; OpenAPI uses `{id}`.

The specification must include:

- an appropriate OpenAPI version,
- an API title and version,
- an HTTPS server URL,
- paths for all required routes,
- path parameters such as the task or report `id`,
- request body schemas for `POST`, `PUT`, and `PATCH` operations,
- successful response schemas,
- common error responses such as `400 Bad Request` and `404 Not Found`,
- a reusable `Task` schema under `components.schemas`.

Security Requirements

The API uses JWT bearer tokens. The specification must also include:

- a reusable security scheme named `bearerAuth` under `components.securitySchemes`,
- `type: http`,
- `scheme: bearer`,
- `bearerFormat: JWT`,
- a security requirement applying `bearerAuth` to protected operations,
- a `401 Unauthorized` response for missing, expired, malformed, or invalid tokens,
- a `403 Forbidden` response for authenticated users who lack permission,
- a brief description of any role restriction, such as an instructor-only operation.

All eight operations listed above must require `bearerAuth`. You may apply the requirement globally or separately to each operation.

15 points

Part 7 — Reflection

10 points

In `answers.md`, answer the following questions. Focus on what you learned and how the topics fit together. Use specific examples, but do not write a novel.

1. Following a Request Through the System

Choose one protected API operation and trace it from the client's request to the server's response.

Explain how at least four of the following participate in processing the request:

- HTTP,
- Express routing,
- middleware,
- authentication,
- authorization,
- database access,
- error handling,
- OpenAPI documentation.

Identify one place where the request could fail and explain how the API should respond.

4 points

2. Synchronous vs. Asynchronous Processing

Describe one operation that should be completed directly in an HTTP request and one operation that would be better handled using a message queue and background worker.

Explain:

- why each processing model is appropriate,
- what the client receives,
- how failures would be handled,
- how a database could be used to track the operation's result or status.

3 points

3. Lessons Learned

Imagine that you must give a short "lessons learned" presentation to a development team building its first web API.

Identify three practices from this course that you would recommend. At least two must relate to topics from the second half of the course, such as:

- database integration,
- password or token security,
- authentication and authorization,
- PKI and HTTPS,
- JWT or OAuth,

- message queues,
- asynchronous processing.

For each practice, briefly explain what problem it prevents or helps solve.

3 points
